Q: Any questions or comments?

Lecture 5: Remote Procedure Calls (RPCs) and Remote Method Invocation (RMI)

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- Remote Procedure Calls & Remote Method Invocation
  - Local vs Remote Calls
  - Interfaces
  - Semantics
  - Transparency
- Java RMI
  - Interfaces
  - Implementation
  - RMI Registry (broker)

Remote Procedure Calls & Remote Method Invocation (Section)

Programming with interfaces

- Many languages support **modularisation**, with programmer-defined **interfaces**
  - The interface defines a **public contract**
- Code external to the module can ONLY use things in the interface
  - i.e. there is **encapsulation**
  - Another programmer needs to understand the interface, not the **implementation**
    - E.g. the module might be implemented in a different **language**
- A module can be **replaced** by another with the same interface and nothing else needs to be changed

Remote Procedure Calls

- The concept that programmers should be able to invoke **remote** operations in the same way as **local procedure calls**
  - first introduced by Birrell and Nelson [1984]
  - E.g. `int doSomething(int a, int b);`
    → Procedure name, parameters, return (typed) value
- The starting point is the programmer's perspective,
  - i.e. code
  - Rather than the network messages or protocol
- But it obviously corresponds to a client/server request-response interaction
  - i.e. call procedure (request) -> result of procedure (response)
  - and should have a well-defined protocol

Local vs remote procedure calls

- In a **distributed** system
  - different **modules** may be running in **different processes**
  - but can still have defined **interfaces**
  - However these may be different to local interfaces…
- In particular, it is **not** (normally) possible for code in one process to access **variables** in another process
  - And on a single machine blocks of **shared memory** *can* be created and shared (by the OS) between processes, but
  - Separate machines have entirely separate physical memory!
  - Emulation of shared memory is possible but slow, depending on the language & system – see later lecture on Distributed Shared Memory

Local vs remote procedure calls (cont.)

- So **call by reference** parameter passing is not possible between processes over networks, because…
- A local variable reference or **pointer** from one process is not meaningful in another processes
  - Local reference = an address in local process memory,
  - Which will point to something different (or illegal) in another process
- So the distributed system always has to **copy** parameters and return values
  - Therefore the RPC system has needs to know if parameters are passed **to** the procedure, returned **from** it, or both
  - (return values and exceptions will always be copied back)

Remote Method Invocation

Q: Has anyone used RPCs or RMI?

- **Remote Method Invocation** applies the concept of RPC to **distributed objects**
  - i.e. code in one process can invoke methods on object that may be in another process
- It also allows **object-oriented** concepts to be used
  - E.g. Inheritance
- It also allows **references** to objects to passed over the network, giving more options for parameter passing
  - Note, a **remote object reference** must identify the host process (IP address and port) as well as the specific object in memory (local address or ID) (so it will probably be an object itself)

Interface definition languages (IDLs)

- RPC & RMI Interfaces may be defined using a language's built-in interface specification facilities
  - E.g. Java `interface`
- Or a separate **interface definition language** may be used
  - Especially if interfaces are language independent
  - E.g. CORBA IDL, Android IDL
- Or built-in interface specification facilities may be used but with additional constraints or annotations
  - E.g. Java RMI – later this lecture/lab

RPC & RMI call semantics

- Local procedure & method call semantics are always **exactly once**
  - i.e. every invoked procedure call is executed exactly once, at least unless the whole process crashes
- Remote procedure & method calls can have different semantics
  - **Maybe** semantics – executed once or not at all
    - May be useful if reliability is not required
  - **At-least-once** semantics – the caller receives a result or an exception is raised
    - The calling process will re-send requests; the operation might be executed more than once; OK if it is idempotent
  - **At-most-once** semantics – the caller receives a result or an exception is raised
    - The calling process will re-send requests; the result of the operation will be saved and re-sent if needed; the operation is never run more than once, but might have run just before a server crash

Transparency

- RPCs & RMI are good examples of **access** (and **location**) **transparency**:
    - (see “other” distributed systems challenges)
  - The user of a module calls procedures in the same way whether the module is local or remote = “access transparency”
- But in practice this transparency is often limited
  - Remote operations can **fail** in more and different ways
  - Remote operations are typically much **slower**
    - e.g. 1000 times slower
- So most RPC/RMI systems will strongly resemble local procedure or method calls but with some additional elements
  - E.g. network-related exceptions, perhaps the option to cancel long-running requests

Java RMI (section)

Java RMI

- **Java RMI** is a Java-specific **Remote Method Invocation** (RMI) middleware
- Many similarities with other distributed object systems
  - E.g. CORBA
- Specific to the Java programming language, e.g.
  - uses Java code (interfaces) to specify remote interfaces
  - Uses Java-specific network encoding
  - So will only work between Java processes

Defining a remote interface

- What is the service?
  - Each service = 1 remote interface
- What distinct requests can a client make?
  - Each request = 1 method
- How can each request by concisely described?
  - = method name
- What information does the client have that the server needs to perform the request?
  - = arguments – types (and names)
- What might be anticipated to go wrong?
  - = exceptions
- If it works, what information does the client need back?
  - = return type

Remote Interfaces in Java RMI

- A Java RMI remote interface is defined using Java's `interface` mechanism, but
  - Must **extend** `java.rmi.Remote`, which marks it as remote
  - Each method must **throw** `java.rmi.RemoteException`, which will signal any network-related failures
  - **Arguments** and return value must be serializable, i.e. primitive types, array, objects implementing `java.io.Serializable` or objects implementing a remote interface (`java.rmi.Remote`)
    - (see following slide)
  - Arguments are only ever sent **to** the remote object (cf. CORBA "in"); only the return value is returned to caller
- For example:

Ex.: Java Remote interfaces *Mailbox*

Remote interface, i.e. API

```
import java.rmi.*;
public interface Mailbox extends Remote {
        public Vector<Header> list()
                throws RemoteException;
        public Message getMessage(int msgid)
                throws RemoteException;
        public void post(Message message)
                throws RemoteException;
}
```

Serialisable classes, for arguments & return values

```
public class Header
implements Serializable {
        public int id;
        public String subject;
}
public class Message
implements Serializable {
        public Header header;
        public String body;
}
```

Java RMI argument & return types

Java RMI arguments and return values can be:

- **primitive** types, e.g. ‘int’,
  - are converted to a standard external representation and sent to the other process
  - which recreates the same primitive value
- (references to) Instances which implement java.io.**Serializable**, e.g. ‘Header’ or ‘Message’
  - are converted to a standard external representation
  - which creates a new object in the remote process with the **same class** and the **same field values**

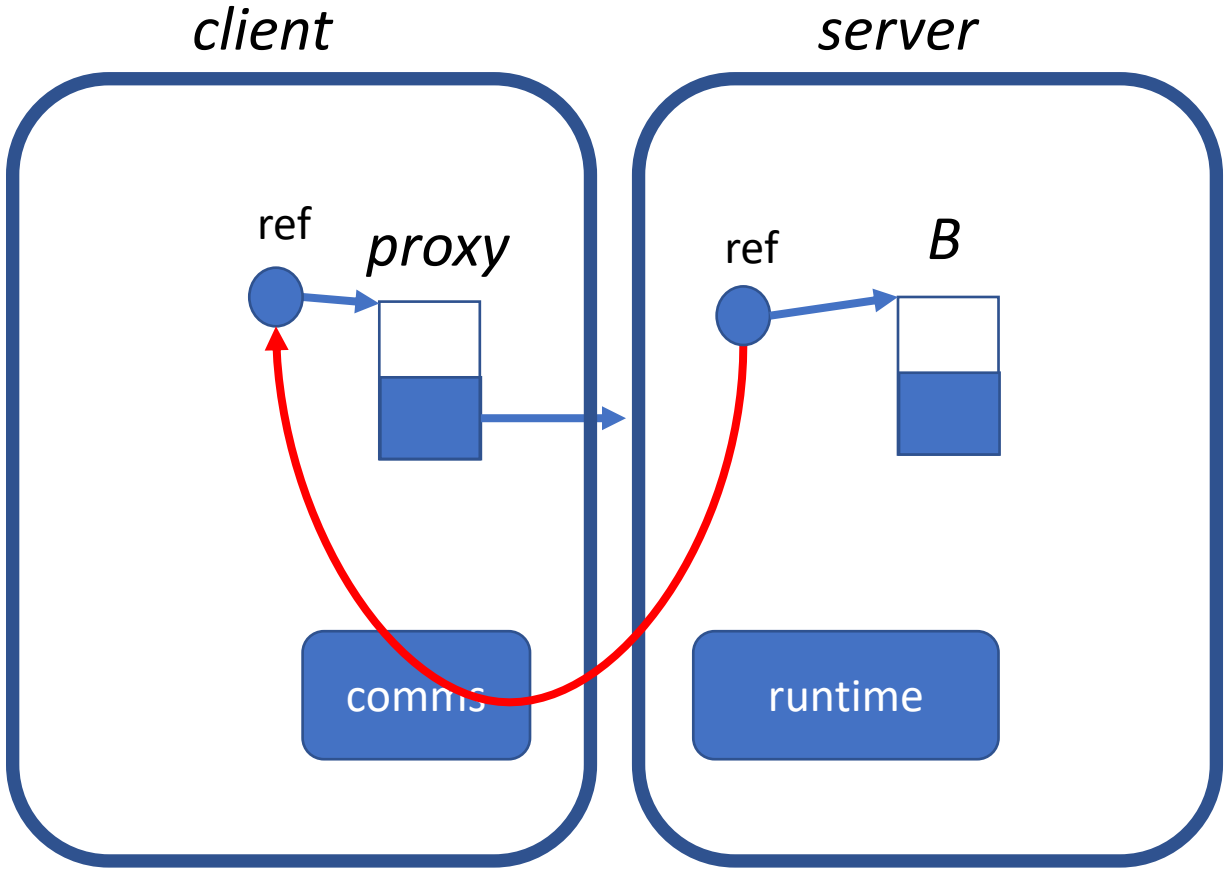
Java RMI argument & return types (cont.)

- (references to) Instances which implement java.rmi.**Remote**, e.g. ‘Mailbox’
    - (see also ‘Notify’ & ‘Mailbox2’ interfaces in Lab 3)
  - cause a new **remote reference** to be created and sent to the remote process…

Implementing RMI

Q: Who knows what reflection is (in programming languages)? Who has used it?

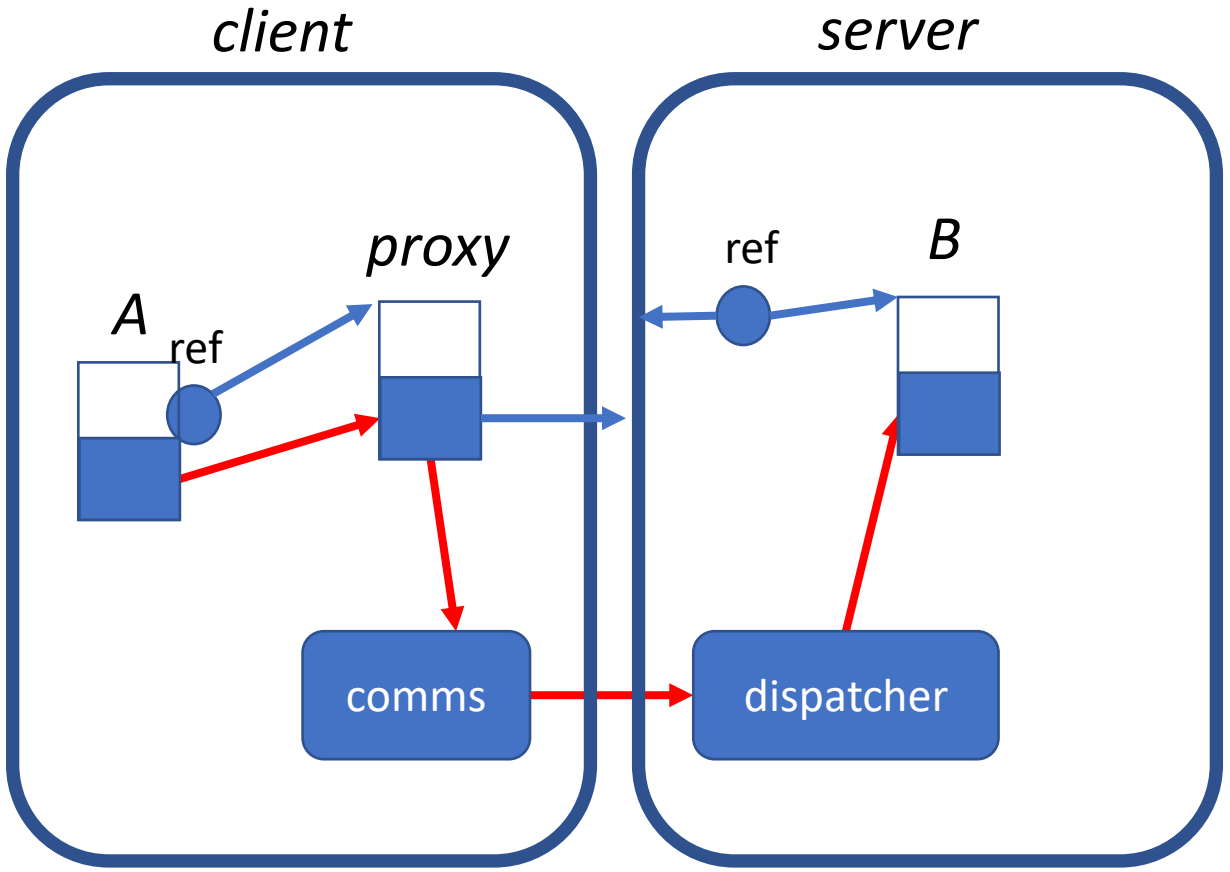
- Consider an object, e.g. *B* in the *server*
- A **remote reference** to *B* must identify both the process and the specific object
  - E.g. *server* IP address, port number, *B*'s local ID
- If a remote reference is passed to another process (e.g. *client*) then it will create a **proxy**[1] object in that process



[1] https://docs.oracle.com/javase/8/docs/api/java/lang/reflect/Proxy.html

Implementing RMI (cont.)

- Object *A* (in the *client*) makes a local method invocation on the proxy
  - The proxy has the same interface but passes invocations to the communication module
- The process hosting the original object has a shared **dispatcher** for the objects it hosts
  - It translates incoming network invocations to local invocations on the original object (*B*)



Implementing a Java Remote Object

- Each remote interface needs an **implementation** that will be run in the server process
- In Java RMI the implementation class **extends** *java.rmi.server.UnicastRemoteObject*
  - Thereby inherits everything it needs to be a remote object
  - By default the dispatcher accepts TCP connections on a new random port, but a specified port can be used
- And **implements** the remote interface, e.g. ‘Mailbox’
  - With (private or protected) instance state
  - And implementations of all of the interface’s (public) methods

Java RMI Registry:
How does the client find a remote object?

- RMI provides a standard **broker** capability called a Registry
  - It runs on a well-known port
- It can be part of the server process itself (as in lab 3) or a separate (potentially shared) **rmiregistry** process
- It maintains a **mapping** from user-friendly names to remote object references - see following lecture on naming
- Remote objects are looked up using URL-like strings of the form:

  `//HOSTNAME:PORT/OBJECTNAME`
  - This is the host and port that the registry is running on
    - Hostname defaults to localhost and port defaults to 1099
  - And the name of the object in that registry

The *Naming* class & *Registry* interface

Class `java.rmi.Naming` provides a static client API, matching the `java.rmi.registry.Registry` interface, both with methods:

*void **rebind** (String name, Remote obj)*

Used by a server to **bind** (associate) a name to a remote object reference.

*void **bind** (String name, Remote obj)*

Similar, but if the name is already bound then an exception is thrown.

*void **unbind** (String name, Remote obj)*

Removes a name binding.

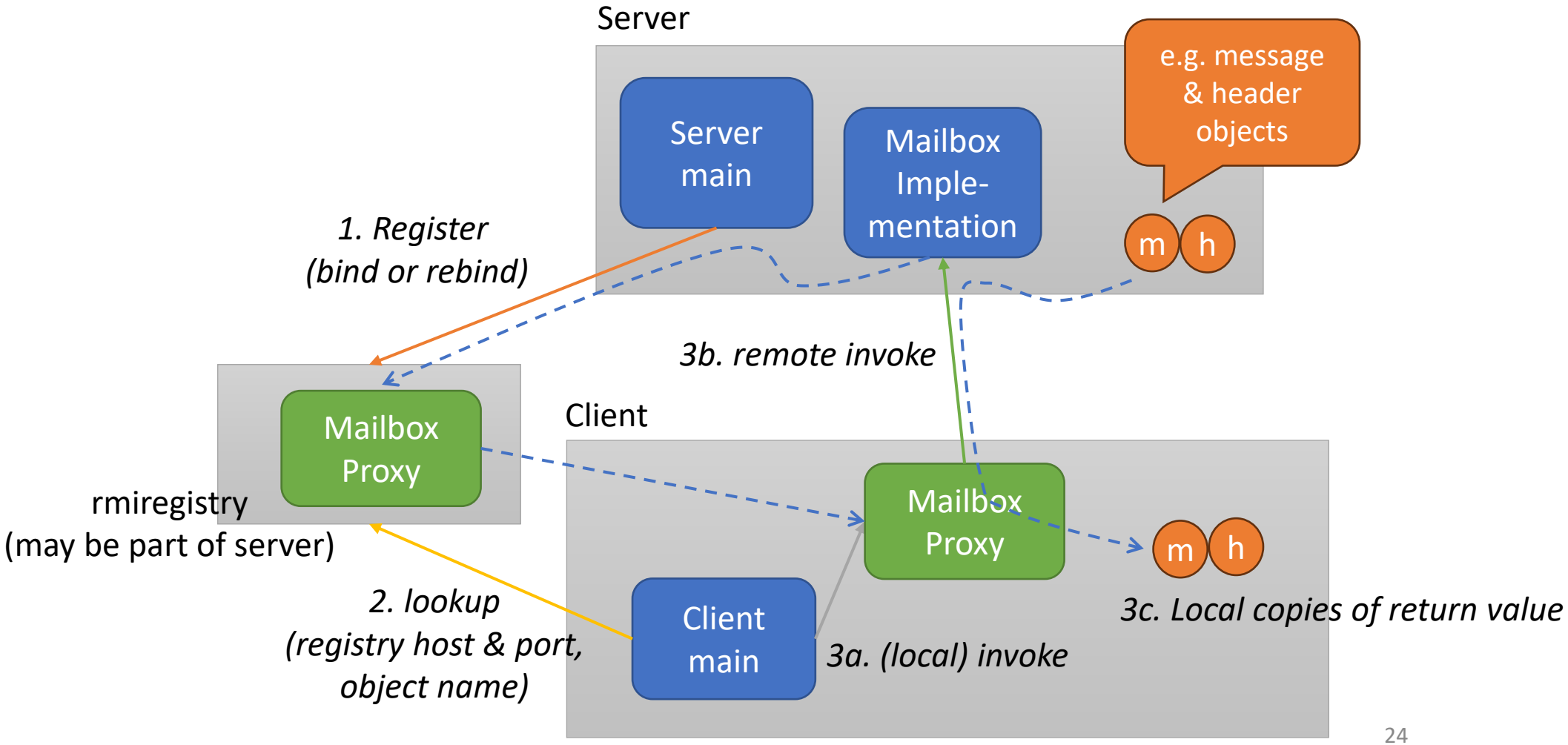
*Remote **lookup** (String name)*

Used by clients to look up a remote object reference by name. A proxy object is returned.

*String [] **list** ()*

Returns an array of all the names bound in the registry.

Java RMI Registry example (e.g. get message)



Remote Procedure Call Summary

- The **Remote Procedure Call** paradigm is the concept that programmers should be able to invoke **remote** operations in the same way as **local procedure calls**
- RPCs are typically defined as an **interface**, perhaps expressed in an **interface definition language**
- RPCs differ from local calls because **memory** is not shared between the caller and the procedure body
  - E.g. no call by reference, no shared variables, explicit in/out annotations of parameters
- RPCs are also much **slower** than local calls and subject to network and server **failures**
  - So in practice access transparency is limited

Remote Method Invocation Summary

- **Remote Method Invocation** (RMI) extends this concept to distributed objects
- A **remote reference** represents a remote object
  - Including the server process host and port, and a specific identifier for the object
- **Java RMI** is a Java-specific **Remote Method Invocation** (RMI) middleware

Java RMI Summary

- A remote interface is a Java **interface** that extends **java.rmi.Remote**
  - Interface types can be **primitive**, **serializable** or **remote**
  - Remote methods throw **java.rmi.RemoteException**
- The implementation class extends **java.rmi.server.UnicastRemoteObject**
- When a reference to a remote object is passed to another process the runtime system creates a **proxy** object in the new process
  - The proxy presents the same interface but converts local invocations in network requests
- A Java **RMI registry** provides a standard **broker** facility to locate remote server object via the **Naming** interface.

Next steps

- Lab 3: Java RMI
- Lecture 6: Name Services and DNS
- (then Q&A and on to WWW)